

Barcode & Expiry Capture: Benchmark Report

Component selection for the expiry-date capture system · 58 real product photographs · 02 September 2026

Summary

Two barcode decoders and one OCR engine were benchmarked against 58 photographs of JustDogs stock, taken by hand on a phone in store conditions. **zxing-cpp is the clear choice for barcode decoding**, reading 29 of the 31 images that actually contain a fully framed barcode (94%) in ~37ms each.

The headline rate across all 58 images is only 50%, but that number is misleading and should not be quoted on its own: **22 of the 58 photographs contain no product barcode at all**, and a further 5 have one sliced by the edge of the frame. The dominant failure mode in this dataset is how the photograph was taken, not how it was decoded.

Headline results

Question	Answer	Evidence
Which barcode library?	zxing-cpp	29/31 vs 21/31 for cv2.barcode
What input resolution?	Resize to 1600px	Best of six sizes tested; native is worse and 5x slower
Add a barcode detector model to crop the ROI first?	No	Cropping to the barcode gained nothing
Can it tell when there is no barcode?	Yes, reliably	0 false positives across 22 no-barcode images
Is OCR a safe fallback for the printed digits?	Only with a catalogue lookup	5/29 recovered, 1 of them wrong

Method

Dataset. 58 JPEG photographs (9 in samples1/, 49 in samples2/), 4032×3024 phone camera, handheld, mixed shelf and desk lighting. Packaging types include foil pouches, rigid jars, PET bottles, cardboard cartons and 1 kg bags. Expiry ground truth for samples2/ comes from the supplied mapping document.

Environment. Apple Silicon macOS, CPU only, no GPU. Python 3.14, zxing-cpp 2.3.0, OpenCV 5.0, RapidOCR (PP-OCRv4 ONNX). Timings are single-threaded wall clock, decode call only, excluding image load.

Classification. Every image that no decoder could read was inspected by eye and labelled as one of: barcode fully in frame, barcode cut off by the frame edge, or no product barcode present. This split is what makes the accuracy figures meaningful, and it is recorded in ground_truth.py so results stay reproducible.

Dataset composition

Category	Count	Share	Why it matters
Barcode fully in frame	31	53%	The only fair denominator for decoder accuracy
Barcode cut off by frame edge	5	9%	Undecodable by design; EAN also printed as text
No product barcode in photo	22	38%	Expiry panel photographed; barcode on another face
Total	58	100%	

Test 1 — zxing-cpp vs cv2.barcode

Both decoders were run over identical 1600px grayscale input. `test_01_decoder_comparison.py`

Subset	zxing-cpp	cv2.barcode
Images with a fully framed barcode (n=31)	29/31 (94%)	21/31 (68%)
Barcode cut off by frame edge (n=5)	0/5	0/5
No barcode present (n=22) — false positives	0/22	0/22
All images (n=58)	29/58 (50%)	21/58 (36%)
Average decode time	37 ms	11 ms

zxing-cpp strictly dominates. It decoded 8 images that cv2.barcode missed; cv2.barcode decoded 0 that zxing-cpp missed. Every cv2 success is a subset of zxing's. cv2 is roughly 3x faster, but 37ms is immaterial at warehouse volumes, so the accuracy difference decides it.

Both are trustworthy about absence. Neither decoder ever reported a barcode in the 22 photographs that contain none. An empty result from `read_barcodes()` is a reliable "no barcode here" signal and needs no extra check.

Test 2 — input resolution, and does cropping help?

The same decoder was run at six input sizes. In addition, for every image where a barcode was located, the region was cropped from the *native* pixels with 8% padding and decoded again — simulating exactly what a detect-then-crop pipeline would feed the decoder. `test_02_resolution_and_crop.py`

Input	Decoded	Rate	Avg time
800 px	15/58	26%	8 ms
1000 px	23/58	40%	13 ms
1200 px	27/58	47%	14 ms
1600 px	29/58	50%	22 ms
2000 px	28/58	48%	31 ms
Native (4032 px)	27/58	47%	108 ms
Cropped to the barcode	28/58	48%	1 ms *

* decode time only; excludes the cost of locating the barcode in the first place.

A barcode detector model is not needed. Cropping to the barcode decoded 28/58 — no better than simply resizing the whole image to 1600px (29/58). zxing-cpp already localises internally: it returns the barcode's corner coordinates, and in this dataset it found one occupying **0.3% of a 12-megapixel frame**. A detect-then-crop stage would add a model dependency and latency for zero measurable gain.

Two further points. **More pixels is not the lever** — native resolution decoded fewer images than 1600px and took five times longer, because downscaling suppresses print texture and glare that confuse the scanline reader. And **multi-scale retry is pointless**: the union of all six resolutions is the same set of images that 1600px alone decodes.

Test 3 — can aggressive retries rescue the failures?

Each of the 29 failing images was retried at native resolution, then with a 4×4 grid of overlapping native-resolution tiles (cropping without a detector), then at 2× upscale. `test_03_failure_recovery.py`

Recovered: 0 of 29. No image was rescued by extra resolution, tiling or upscaling. This is the strongest evidence in the report that the remaining failures are not a decoder problem. Retrying cannot invent pixels that were never captured — the fix belongs at capture time.

Test 4 — OCR of the printed digits as a fallback

When the bars are unreadable, the human-readable digits are often still legible, and on the cut-off packs the full EAN appears in the label text next to the expiry. Each failing image was OCR'd and every 13-digit run validated with the EAN-13 check digit. `test_04_ocr_barcode_fallback.py`

Outcome	Count
Failures attempted	29
Recovered a checksum-valid EAN-13	5
Of those, demonstrably WRONG	1

A check digit is a 1-in-10 filter, not proof. For samples1/20260829_192628.jpg OCR produced 8904505198366, which passes EAN-13 validation, while the true code is 8904505983665. Roughly one in ten corrupted reads will pass by chance. **Any OCR-recovered barcode must also be looked up in the product master before it is trusted;** if the code is not in the catalogue, reject it.

Used carefully the fallback still earns its place. Where OCR drops only the leading digit (which sits outside the guard bar), brute-forcing the missing digit against the check digit resolves it uniquely:

```
OCR saw 12 digits '904505983665'  
→ checksum-valid completions: ['8904505983665'] (exactly one)
```

Test 5 — expiry date OCR

RapidOCR (PP-OCR models on onnxruntime) was benchmarked against the 44 images carrying a ground-truth expiry date. Scoring asks only whether the correct date appears anywhere in the raw OCR output, in any form the pack might print it — no parsing, no normalisation — so the number measures the engine rather than a regex. Full suite in tests/ocr_expiry/.

Input resolution

Input	Correct	Rate	Avg time
1000 px	31/44	70%	418 ms
1600 px	29/44	66%	596 ms
2200 px	28/44	64%	810 ms
3000 px	28/44	64%	1083 ms
Native (4032 px)	24/44	55%	1711 ms

As with the barcode, the lowest resolution wins and native is both worst and four times slower. Downscaling suppresses packaging texture and gloss that otherwise generate spurious text detections.

Preprocessing and cropping

Strategy	Correct	Rate
Whole image @1000px (baseline)	31/44	70%
Whole image + unsharp mask	33/44	75%
Two-pass: OCR-located box, re-OCR	31/44	70%
Crop to hand-drawn date panel	25/44	57%
Crop to date panel, native resolution	24/44	55%
Adaptive threshold (binarise)	19/44	43%

Cropping to the expiry region makes OCR worse, not better. Even with the date panel located by hand on every image and each crop verified by eye, accuracy fell from 31/44 to 25/44. Modern OCR is two-stage: a detector finds text regions and a recogniser reads them, so the crop already happens inside the model. Cropping to ~24% of the frame pushes the input outside the distribution the detector was trained on — on sample 1 it found 56 text regions in the whole photo but only 7 in the crop, missing the MFG and EXP lines entirely even though nothing was clipped. Do not build a date-region detector: a perfect one would still lose accuracy.

The same applies to classical preprocessing. Adaptive threshold — recommended in the original architecture proposal — destroyed 12 of the 31 images that worked untouched, and CLAHE lost 3. Both are correct advice for Tesseract, which needs binarised input, and wrong for RapidOCR, whose models are trained on natural colour photographs. Only mild sharpening helped: **+2 images, none lost, no extra cost.**

Recommended pipeline: `exif_transpose` → `resize 1000px` → `unsharp mask` → `RapidOCR` — 33/44 in ~420 ms, no cropping, no threshold.

Measure the engine, not the regex. A strict DD/MM/YYYY parser scores only 22/44 on the same OCR output that contains the correct date 31 times. Sample 39 returns `EXP2028 . JAN . 21` — exactly right, but it matches no conventional date pattern. Four formats appear across the packs (04/12/2026, 20250110, 10/2027, January 2027), and sideways text needs a rotation retry: upright OCR returned `EPDATE210/2027` where 90° returned `EAPDATE23 ./10/2027`. The date parser, not the OCR engine, is now the weakest component.

Findings that affect the design

1. The bottleneck is capture, not extraction

Of the 29 images that failed to decode, 22 contain no barcode at all and 5 have one cut off by the frame edge — 27 of 29 are framing problems. Only 2 are genuine decode failures (both curved jars, one also out of focus). Effort spent tuning the decoder will return almost nothing; effort spent on how the photograph is taken will return almost everything.

2. Some packs cannot be captured in a single photograph

On bottles, jars and cartons the barcode and the expiry panel are frequently on opposite faces. The supplied ground-truth document confirms this directly: samples 46 and 47 are the same product, and the note states the expiry is only derivable by combining both images. The one-photo assumption holds for flat pouches and bags but breaks for rigid containers, so the workflow needs a second-photo path rather than treating it as an error.

3. Multiple barcodes in one frame is a real case

Sample 42 contains **six** distinct, individually checksum-valid EAN-13 codes (a shelf or multipack shot); sample 38 contains two. The two decoders returned *different* products for sample 42 — both legitimately present. The system must not silently take the first result: when more than one retail barcode decodes, the employee should be asked which product they are capturing.

4. Not every barcode is a product barcode

Sample 25 is a hang-tag carrying a QR code (op691ab19bbf18b) and no EAN. Results must be filtered to EAN13 / EAN8 / UPCA / UPCE, or a QR payload will end up written into a `product_id` field.

5. Expiry is not always printed

Several packs print only a manufacture date plus a shelf-life statement ("best before 24 months from date of manufacture"), so expiry must be derived per SKU. On one pack the supplier printed the labels "Best Before:" and "Batch No:" and left both blank — OCR is correct and the data simply does not exist. Both cases need a defined path, not an error state.

Recommendations

- **Adopt zxing-cpp for barcode decoding.** Resize to 1600px with EXIF orientation applied, filter results to retail formats, and do not add a detector model or a cropping stage.
- **Treat an empty result as a definite "no barcode",** and surface it to the employee immediately rather than uploading and failing later.
- **Scan live from the camera preview rather than from a single still.** This is the highest-value change available: retrying every frame structurally eliminates both the cut-off and the out-of-focus failures, and blocks submission until a code actually decodes.
- **Gate every OCR-recovered barcode on a product-master lookup.** The check digit alone admits roughly one wrong code in ten.
- **Handle multi-barcode frames explicitly** by asking the employee to choose, and add a second-photo path for rigid containers where the barcode and expiry are on different faces.
- **Invest the next block of effort in the date parser,** which is now the weakest component — all four observed formats, a rotation retry, and shelf-life-derived expiry for packs printing only a manufacture date.

Reproducing these results

```
cd backend/tests
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python run_all.py # all five tests, logs to results/
python run_all.py 01 02 # selected tests only
python make_report.py # regenerate this PDF
```

Each test writes a JSON file of per-image results into `results/`, and this report reads its figures back from those files so the document cannot drift from the measurements.